

# Remake\_Project 진행

정혜교

# Unity 씬 구성

- NetworkSystem
  - └── NetworkManager
  - └── UnityTransport
  - └── NetworkLauncher

# 현재까지 한거

- 멀티플레이 패키지와 Unity Cloud 프로젝트를 연결 준비
- Unity Transport, Netcode for GameObjects, Multiplayer Services, Multiplayer Tools, Multiplayer Center 등 패키지 설치
- 매니저 참조만 연결, 실제 동작 없음

# 전체 구조

- NetworkManager
  - └─ 네트워크 연결과 세션 자체를 관리
- NetworkObject
  - └─ 특정 GameObject를 네트워크 객체로 등록
- NetworkMatchBridge
  - └─ 네트워크 요청·상태와 기존 게임 매니저를 연결
- GameManager
  - BattleManager
  - PlayerManager
    - └─ 실제 게임 규칙과 데이터 처리
- InputManager
  - └─ 현재 컴퓨터의 플레이어 입력을 처리

# NetworkManager

- Host 시작
- Client 시작
- Dedicated Server 시작
- 접속한 플레이어 관리
- 연결 종료 관리
- Network Prefab 관리
- NetworkObject Spawn 관리
- 네트워크 씬 관리
- Transport 관리
- 네트워크 시간과 Tick 관리

# NetworkManager

- 방장이 방을 만들 때 : `NetworkManager.Singleton.StartHost();`
- 참가자가 들어갈 때 : `NetworkManager.Singleton.StartClient();`

# NetworkObject(공용 객체)

- NetworkObjectId
- OwnerClientId
- IsSpawned
- IsOwner
- IsLocalPlayer
- IsPlayerObject

# NetworkMatchBridge(중계 담당)

- 클라이언트의 요청을 서버로 전달
- 서버에서 게임 행동을 검사
- GameManager에 게임 시작 또는 종료 요청
- BattleManager에 공격 처리 요청
- PlayerManager에 플레이어 상태 처리 요청
- 서버의 결과를 모든 클라이언트에 동기화(이게 제일 중요)



# 그 이후 해야 하는 절차

- 1단계 : 프로젝트와 패키지 환경 통일
- Unity Editor 버전
- Netcode for GameObjects 버전
- Unity Transport 버전
- Multiplayer Services 버전
- manifest.json
- packages-lock.json
- Build Profiles 또는 Build Settings의 씬 목록
- Input System 설정
- 같은 Unity Cloud Project 사용 여부

- 2단계 : NetworkManager 씬 구성(이거 함)
- 3단계 : Host와 Client 실행 코드 작성(NetworkLanucher 작성하기)
- 4단계 : Player Prefab 네트워크화
- 5단계 : NetworkMatchBridge를 네트워크 오브젝트 구성
- 6단계 : 게임 상태 동기화
- 7단계 : 게임 시작 RPC 작성
- 8단계 : 접속과 종료 처리
- 9단계 : 이동 동기화
- 10단계 : 전투 동기화

# Unity 멀티플레이 구조도

